GeneScript: A Biology-Inspired Programming Language

Compiler Construction Project - final submission (Phase 1 and Phase 2)

Course	Compiler Construction (BCS-7G)
Instructor	Sir Faisal Aslam
Phase 1	Interpreter (Flex and Bison)
Phase 2	Ahead-of-time compiler (LLVM IR, compiled with Clang)

Team Member	Roll Number
Eman Aamir	23L-0612
Haleema Sadia	23L-0960
Ahsan Naveed	23L-0756

---

1. Project Overview and Language Purpose

GeneScript is a small, beginner-friendly language for bioinformatics students. In most languages DNA and RNA are plain text, so a typo such as `"ATXG"`, or RNA used where DNA belongs, runs without complaint and only shows up later as a wrong result. GeneScript makes `dna` and `rna` built-in types and checks them before the program runs, with error messages that state the biological rule that was broken.

Key ideas from the proposal (`docs/GeneScript_Proposal.pdf`):

- **Built-in DNA and RNA types.** DNA may contain only A, C, G, T; RNA only A, C, G, U. Any other letter, or a mix of the two alphabets, is a compile-time error.
- **Biology keywords.** A function is a `gene`, `express` returns a value, `report` prints, blocks end with `stop`, and the program starts at `gene nucleus()`.
- **Value-first syntax.** `"ATGC" -> dna g` declares a variable; `tail(g) -> g` assigns.
- **Built-in operations.** `head(s)`, `tail(s)`, `s@i`, `#s`, `fuse a b`, `~d` (reverse complement) and `transcribe(d)`.
- **Biological data only.** Every value that is stored, passed, returned or printed is a DNA or RNA sequence. Integers exist only as loop bounds, positions (`s@i`), `#s` and loop counters.

Concept	GeneScript
Function / return	`gene name(dna a, rna b) ... stop`, `express e`
Entry point	`gene nucleus()` (exactly one, no parameters)
Declaration / assignment	`e -> dna x`, `e -> rna x`, `e -> x`
Output	`report e`
If / else if / else	`when c ... otherwise when c ... otherwise ... stop`
Loops	`cycle i across s`, `cycle i from a to b`, `grow until c`
Break / continue	`halt` / `skip`
Logic	`both`, `either`, `lacks`, values `bound` / `unbound`
Comparisons	`=` (whole sequences), `< > <= >=` (single bases only)
Literals	`"ATGC"`, `""` (empty), `'G'` (one base)
Comment	`^^ until the end of the line`

2. Phase 1: Interpreter Architecture

```
source.gene -> Lexer (Flex) -> Parser (Bison) -> AST -> Semantic analysis -> Interpreter
               src/lexer.l     src/parser.y           src/semantics.cpp     src/interpreter.cpp
```

1. **Lexical analysis** (`src/lexer.l`). Turns the source into tokens: keywords, operators, sequence and base literals, integers, identifiers and `^^` comments. An unknown character is reported as `file:line: unrecognized character` and stops compilation.
2. **Parsing** (`src/parser.y`). A Bison LALR(1) grammar for value-first declarations, prefix `fuse`, the `when` / `otherwise when` chain (one `stop` per chain), the loops and gene definitions. It builds the abstract syntax tree defined in `include/ast.h`.
3. **Semantic analysis** (`src/semantics.cpp`), in three passes:
   - collect every gene's signature and reject duplicate gene names, so a gene can call any gene in the file, before or after it;
   - infer each gene's return type from all of its `express` statements (including ones nested in conditions and loops), repeating until nothing changes, so recursion and mutual recursion are typed correctly;
   - check every gene body: literal alphabets, DNA/RNA type rules, argument types, the integer rules (`#s` only as a loop bound, `s@i` positions, `cycle` bounds), conditions, `halt` / `skip` placement, and exactly one `nucleus()`.

   Scopes are handled by `include/symbol_table.h`. All errors are collected and printed in the form `file:line: message` followed by an explanatory note line.
4. **Interpretation** (`src/interpreter.cpp`). A tree-walking interpreter evaluates the AST directly in memory: there is no intermediate bytecode. Execution starts at `nucleus()`.

3. Phase 2: LLVM Ahead-of-Time Compiler Architecture

```
source.gene -> same front end as Phase 1 -> CodeGenerator -> program.ll -> clang -> native executable
               (lexer, parser, semantics)   src/codegen.cpp              + runtime/gs_runtime.c
```

- **Shared front end.** The compiler uses exactly the same lexer, parser and semantic analyzer as the interpreter, so both report identical errors, and no IR is written for an invalid program.
- **Code generation** (`src/codegen.cpp`). Walks the checked AST and writes textual LLVM IR (a `.ll` file). `IRContext` handles functions, basic blocks, unique labels, temporaries and scopes; `CodeGenerator` lowers each GeneScript construct.
- **Value representation.** Every DNA/RNA value is a pointer (`ptr`) to an immutable, reference-counted `GsSeq { refcount, len, kind, data }`. Literals are constant globals. Conditions are `i1` values and integers are `i64`.
- **Genes.** Each gene becomes `define ptr @gs_gene_<name>(ptr ...)`. A generated C `main` calls `@gs_gene_nucleus`. Calls to genes defined later in the file need no declarations, so recursion and mutual recursion work directly.
- **Control flow** uses LLVM basic blocks and `br i1` branches: `when.then.N`, `when.else.N`, `when.end.N` for conditions; `cond` / `body` / `step` / `end` blocks for loops; `halt` and `skip` branch to the loop's end and step blocks; `both` / `either` short-circuit through a `phi`.
- **Runtime** (`runtime/gs_runtime.c`). A small C library for the sequence operations, `report`, and reference counting. `genec -c` compiles it together with the IR.
- **Driver** (`src/genec.cpp`). The `genec` command runs the interpreter, writes IR, or calls Clang to build an executable.

The full ABI (value layout, calling convention, ownership rules) is in `docs/LLVM_BACKEND.md`.

4. Dependencies and Installation (Ubuntu / Linux)

Package	Needed for
`build-essential` (g++, make)	building everything
`flex`, `bison`	generating the lexer and parser
`clang`	`genec -c` ahead-of-time compiler and compiler tests
`llvm`	`llvm-as`, used by compiler tests to verify IR (optional)
`python3-reportlab`	generating `README.pdf` from markdown (`make pdf`)

```
sudo apt update
sudo apt install -y build-essential flex bison clang llvm python3-reportlab
```

(Alternatively, if using Python virtual environments: `pip install reportlab`)

The project was verified on Ubuntu 24.04 and 26.04 LTS with g++ 13/15, flex 2.6.4, GNU Bison 3.8.2, GNU Make 4.4.1 and Clang 18/21.

On Windows, use WSL (Ubuntu) and run every command inside the Ubuntu terminal. The programs built by `make` are Linux executables and cannot run from Git Bash or PowerShell.

5. Building the Project

Run these from the project root:

Command	Result
`make`	builds everything: `bin/genescript`, `bin/genec` (copied to `./genec`) and `bin/genescript-llvm`
`make interpreter`	builds only the Phase 1 interpreter `bin/genescript` (no Clang needed)
`make compiler`	builds only the Phase 2 tools `bin/genec`, `./genec` and `bin/genescript-llvm`
`make clean`	removes `bin/`, `./genec`, the generated lexer/parser and all `.o` files

Building needs only g++, flex and bison. Clang is needed when compiling GeneScript programs to native code (`genec -c`) and for `make test-compiler`.

6. Running Programs with the Interpreter

```
./genec tests/sample1_filter.gene
```

Without options, `genec` runs the program with the Phase 1 interpreter. `bin/genescript tests/sample1_filter.gene` is the standalone Phase 1 interpreter and behaves identically. The exit status is 0 on success and 1 after a compile-time or runtime error.

```
$ ./genec
Usage: ./genec <source.gene>                run with the interpreter
       ./genec -S <source.gene> [-o out.ll]  write LLVM IR
       ./genec -c <source.gene> [-o prog]    compile to a native executable
```

7. Generating LLVM IR and Native Executables

Command	Result
`./genec -S prog.gene`	writes LLVM IR to `prog.ll` in the current directory
`./genec -S prog.gene -o out.ll`	writes LLVM IR to `out.ll`
`./genec -c prog.gene`	builds the native executable `./prog`
`./genec -c prog.gene -o name`	builds the native executable `./name`

For example:

```
./genec -c tests/sample2_mergesort.gene -o mergesort
./mergesort
```

`genec -c` writes the IR to a temporary file and runs `clang -O2 <tmp>.ll runtime/gs_runtime.c -o <name>`. It finds `runtime/gs_runtime.c` next to the `genec` executable (or one folder above it, for `bin/genec`). Two environment variables change this: `CLANG` selects the compiler (default `clang`) and `GS_RUNTIME` gives the path of `gs_runtime.c`. If Clang is missing or fails, `genec` prints a message, exits with status 1 and leaves no executable.

IR written with `-S` can also be compiled by hand:

```
./genec -S tests/sample1_filter.gene
clang sample1_filter.ll runtime/gs_runtime.c -o sample1_filter
./sample1_filter
```

An excerpt of the IR for a small gene, showing a condition as basic blocks and a `br i1`:

```
gene first(dna s)               ; GeneScript source
    when s = ""
        express s
    stop
```

```
    express head(s)
stop

define ptr @gs_gene_first(ptr %p.s) {
entry:
  %v.s.0 = alloca ptr, align 8
  %t1 = call ptr @gs_seq_retain(ptr %p.s)
  store ptr %t1, ptr %v.s.0, align 8
  %t2 = load ptr, ptr %v.s.0, align 8
  %t3 = call i32 @gs_seq_equal(ptr %t2, ptr @.seq.0)
  %t4 = icmp ne i32 %t3, 0
  br i1 %t4, label %when.then.1, label %when.end.0
when.then.1:
  %t5 = load ptr, ptr %v.s.0, align 8
  %t6 = call ptr @gs_seq_retain(ptr %t5)
  %t7 = load ptr, ptr %v.s.0, align 8
  call void @gs_seq_release(ptr %t7)
  ret ptr %t6
when.end.0:
  %t8 = load ptr, ptr %v.s.0, align 8
  %t9 = call ptr @gs_seq_head(ptr %t8)
  %t10 = load ptr, ptr %v.s.0, align 8
  call void @gs_seq_release(ptr %t10)
  ret ptr %t9
}
```

8. Example Programs and Expected Output

Both proposal samples give the same output with the interpreter and as native executables.

Sample 1: filter G and C bases (`tests/sample1_filter.gene`)

```
gene gc_filter(dna seq)   ^^ keeps only the G and C bases of a DNA sequence
    "" -> dna result
    cycle i across seq
        when seq@i = 'G' either seq@i = 'C'
            fuse result seq@i -> result
        stop
    stop
    express result
stop

gene nucleus()
    "ATGGCCATTGTA" -> dna g
    report gc_filter(g)
    report ~g

    transcribe(g) -> rna r
    report r
stop

$ ./genec tests/sample1_filter.gene                    # Phase 1: Interpreter
$ ./genec -c tests/sample1_filter.gene -o sample1      # Phase 2: AOT Compiler
$ ./sample1                                           # Run native executable
GGCCG
TACAATGGCCAT
AUGGCCAUUGUA
```

Sample 2: recursive merge sort (`tests/sample2_mergesort.gene`)

```
gene merge(dna a, dna b)   ^^ joins two sorted sequences into one
    when a = ""
        express b
    stop
    when b = ""
        express a
    stop
    when head(a) < head(b)
        express fuse head(a) merge(tail(a), b)
    otherwise
        express fuse head(b) merge(a, tail(b))
    stop
stop
```

```
gene split_even(dna s)   ^^ bases at positions 0, 2, 4, ...
    when s = ""
        express s
    stop
    express fuse head(s) split_odd(tail(s))
stop

gene split_odd(dna s)   ^^ bases at positions 1, 3, 5, ...
    when s = ""
        express s
    stop
    express split_even(tail(s))
stop

gene merge_sort(dna s)   ^^ sorts the bases of a DNA sequence
    when tail(s) = ""
        express s
    stop
    express merge(merge_sort(split_even(s)), merge_sort(split_odd(s)))
stop

gene nucleus()
    "GATTACA" -> dna g
    report merge_sort(g)
stop

$ ./genec tests/sample2_mergesort.gene                 # Phase 1: Interpreter
$ ./genec -c tests/sample2_mergesort.gene -o mergesort  # Phase 2: AOT Compiler
$ ./mergesort                                          # Run native executable
AAACGTT
```

More examples, one per language feature, are in `tests/compiler/` (for example `recursion.gene`, `mutual_recursion.gene`, `loops.gene`, `break_continue.gene`), each with its expected output in `tests/compiler/expected/`.

9. Error Examples

Every compile-time error is printed as `file:line: message` plus a note line, all errors are listed, and compilation stops with exit status 1. No IR or executable is produced.

Lexical error (`tests/compiler/errors/lexical_error.gene`)

```
gene nucleus()
    "ACGT" -> dna g $
    report g
stop

tests/compiler/errors/lexical_error.gene:3: unrecognized character '$'.
              GeneScript has no such symbol; sequences go in quotes and comments start with ^^.
Compilation stopped: 1 error.
```

Syntax error (`tests/compiler/errors/syntax_error.gene`): a `when` without its `stop`

```
gene nucleus()
    "ACGT" -> dna g
    when g = "ACGT"
        report g
stop

Syntax Error at line 7, col 5: syntax error near ''
Compilation stopped: 1 error.
```

Semantic errors

The proposal's sample (`tests/error.gene`, the single line `"ATXG" -> dna x`):

```
tests/error.gene:1: invalid base 'X' in DNA literal.
              DNA allows only A, C, G, T.
Compilation stopped: 1 error.
```

Mixing DNA and RNA (`tests/llvm/errors/fuse_dna_rna.gene`, line 5 is `report fuse d r`):

```
tests/llvm/errors/fuse_dna_rna.gene:5: cannot fuse dna with rna.
              fuse requires both sequences to have the same biological type.
```

```
Compilation stopped: 1 error.
```

Wrong argument types (`tests/compiler/errors/call_arg_type.gene`):

```
tests/compiler/errors/call_arg_type.gene:8: argument 1 of gene 'only_dna' must be dna, but got rna.
             DNA and RNA are separate types; a parameter accepts only its declared type.
tests/compiler/errors/call_arg_type.gene:9: invalid base 'U' in DNA literal.
             base 'U' is RNA only; DNA allows only A, C, G, T.
Compilation stopped: 2 errors.
```

Other semantic errors covered by tests include: undeclared variables, redeclaration, `#s` outside a loop bound, invalid `s@i` positions and `cycle` bounds, ordering operators on whole sequences, `~` or `transcribe` on RNA, conditions that are not conditions, assigning to a loop counter, `halt` / `skip` outside a loop, wrong argument counts, undefined or duplicate genes, return-type mismatches, a missing or duplicated `nucleus()`, and statements outside any gene.

Runtime error

Ordering operators need single bases. When a variable holds more than one base, the check happens at run time; output printed before the error is kept and the exit status is 1:

```
$ ./genec tests/compiler/runtime_errors/ordering_in_gene.gene
A
Runtime Error (line 3): ordering operators compare single bases only, but got "AC" and "G".
```

10. Source Files and Folders

Path	Description
`src/lexer.l`	Flex scanner: keywords, operators, literals, `^^` comments, lexical errors
`src/parser.y`	Bison LALR(1) grammar; builds the AST
`include/ast.h`	AST node classes, the runtime `Value` type, and `evaluate` / `execute` hooks
`include/symbol_table.h`	Nested scopes and symbols (variables, read-only loop counters)
`include/semantics.h`, `src/semantics.cpp`	Semantic analyzer: signatures, return-type inference, type and biology rules, error messages
`include/interpreter.h`, `src/interpreter.cpp`	Phase 1 tree-walking interpreter
`src/main.cpp`	Driver for `bin/genescript` (Phase 1 interpreter only)
`include/codegen.h`, `src/codegen.cpp`	Phase 2 code generator: AST to textual LLVM IR
`src/genec.cpp`	Driver for `genec`: interpreter, `-S` (IR) and `-c` (native via Clang)
`src/llvm_main.cpp`	Development driver `bin/genescript-llvm` (IR only), used by `tests/llvm`
`runtime/gs_runtime.h`, `runtime/gs_runtime.c`	C runtime linked into every native program
`docs/LLVM_BACKEND.md`	Phase 2 design: runtime ABI, calling convention, ownership, control flow
`Makefile`	Build and test targets (sections 5 and 11)
`generate_pdf.py`	Builds `README.pdf` from `README.md` (`make pdf`, requires `reportlab`)
`CMDS.txt`	Quick terminal reference cheat-sheet for Phase 1 & Phase 2 commands
`tests/*.gene`, `tests/expected/`	Phase 1 tests: the proposal samples, `error.gene`, and regression tests `fix*.gene`
`tests/run_tests.sh`	Phase 1 regression test runner
`tests/llvm/`	Phase 2 backend tests (valid programs, `errors/`, `expected/`), `runtime_unit.c`, runner
`tests/compiler/`	End-to-end `genec` tests: features, `runtime_errors/`, `errors/`, `expected/`, runner
`docs/GeneScript_Proposal.pdf`	The original language proposal
`bin/` (generated)	Built programs and test output; removed by `make clean`

11. Automated Tests

```
make test            # Phase 1: interpreter tests (needs only the interpreter)
make test-compiler   # Phase 2: backend tests + end-to-end genec tests (needs clang)
make test-llvm       # only the backend tests (tests/llvm)
```

- `make test` runs the two proposal samples and `error.gene`, then `tests/run_tests.sh`, which compares the output of each `tests/fix*.gene` program with `tests/expected/`.
- `make test-compiler` runs two suites:
  - `tests/llvm/run_llvm_tests.sh`: a unit test of the C runtime; every `tests/llvm/*.gene` program is turned into IR, verified with `llvm-as`, compiled with `clang -Werror`, run, and compared with its expected output and with the interpreter; every `tests/llvm/errors/*.gene` program must be rejected with the expected diagnostics; and every Phase 1 program in `tests/` must give the same result natively.
  - `tests/compiler/run_compiler_tests.sh`: for each feature program, the interpreter and the native executable must both exit with status 0 and both print the expected output; runtime-error programs must exit with status 1 in both; every invalid program must be rejected by `genec -c` and `genec -S` with exit status 1 and no executable or `.ll` file; every other test program in the project must give the same output and exit status natively as in the interpreter; and the `genec` options and failure cases (missing file, unknown option, Clang missing or failing) are checked.

Native test programs are also built with AddressSanitizer and must run with no memory errors or leaks. Set `GS_NO_ASAN=1` to skip those runs, or `CLANG=clang-18` (for example) to choose the Clang version. Test output is written under `bin/`.

Results of a clean build on Ubuntu 26.04.1 LTS (`make clean && make && make test && make test-compiler`):

Suite	Result
`make test`	14 passed, 0 failed (plus the two samples and `error.gene`, which print the outputs shown in sections 8 and 9)
`make test-compiler`: backend suite	55 passed, 0 failed, 0 skipped (1 runtime unit test, 20 valid programs, 17 error programs, 17 Phase 1 programs)
`make test-compiler`: end-to-end suite	102 passed, 0 failed (11 feature programs, 1 runtime error, 13 invalid programs, 54 regression programs, 23 CLI checks)

12. Known Limitations

- **Syntax error messages** come from Bison and use the form `Syntax Error at line L, col C: ... near 'token'`, without the file name. At the end of the file the token is empty (`near ''`) and the line is one past the last line.
- **Block scoping differs for reused names.** The interpreter keeps one set of variables per gene call, so declaring a variable (or a loop counter) inside a block with the same name as an outer one overwrites the outer one. The compiled program uses block scope, as the proposal describes for counters. The two give different output only for such programs; no test program reuses a name this way.
- `report` **of a condition** (for example `report g = "A"`) is accepted and prints an empty line in both the interpreter and native code (kept from Phase 1). The proposal says only sequences are printed.
- **Very deep recursion** has no depth check. The interpreter uses much more stack per call than native code: a gene that recursed 18,000 levels deep crashed the interpreter (segmentation fault, exit status 139) while the compiled program ran correctly. Native programs can also run out of stack, only at a far greater depth.
- `genec -c` **needs Clang installed** and `runtime/gs_runtime.c` next to `genec` (or the `GS_RUNTIME` environment variable). Executables are produced for Linux only.
- **Default** `genec` **mode.** `./genec file.gene` runs the interpreter and `-c` compiles. The proposal's screenshots show compiling with `./genec sample.gene && ./sample`; with this implementation the equivalent is `./genec -c sample.gene && ./sample`.
- **Integers** cannot be declared, stored, compared, passed, returned or printed, by design of the language (see section 1).